

DC-LAB 1: Creating a Synthesis Flow with DC

- In DC-LAB 0, you already copied the “FE” folder to your home directory, so now go to the LAB1/ folder:

```
UNIX% cd ~/FE/LAB1
```

- Let’s write the **synthesis flow script** for the Registered Mux-4 design from DC-LAB 0, which you can later recycle for the CPU (microprocessor) you implemented.
- In the “scripts” folder in “LAB1”, you will find the TCL file “dc.tcl” with the sections of the Design Compiler synthesis flow. Add the Design Compiler commands that correspond for each step:
 1. Search Path and Logic Library Setup
 2. RTL Reading and Link
 3. Constraints Setup
 4. Pre-compile Reports
 5. Compile/Synthesis
 6. Post-compile Reports
 7. Save Design
- Tips:
 - ✓ You can work in interactive mode first, and as you verify your commands are correct add them to the “dc.tcl” script to run in batch mode.
 - ✓ You can add a return command at any point of the DC script to make it stop there while running in batch mode and return to the dc_shell in interactive mode.
 - ✓ **Always check the messages printed on screen by Design Compiler after each command you have written.** They give important information, warnings and errors that will help you understand how the flow is developing and to catch any errors in your script.
 - ✓ You can use a .csh file to automatically run Design Compiler with your script. In your Linux shell, run the “run.csh” file in LAB1/. You can review the script with the “cat” command or using a text editor. It should have the following line that loads the DC shell, sources your “./scripts/dc.tcl” run script and saves the output to “logs/dc.log”:
 - dc_shell -f ./scripts/dc.tcl | tee -i ./logs/dc.log

Part A: Technology configuration and RTL reading

1. Search Path and Logic Library Setup & 2. RTL Reading and Link

Use the following information:

Search path	\$search_path . ./rtl ./libs
Target libraries	sky130_fd_sc_hd__ff_100C_1v95.db sky130_fd_sc_hd__ss_100C_1v40.db
Link library	* \$target_library
RTL	top.v mux4_registered.v mux4.v register_bank.v

(Basically, copy the script from LAB0)

Part B: Constraining the design

3. Constraints Setup

- Set-up the following timing constraints using the appropriate Design Compiler commands. Work with the “dc_shell” in interactive mode first while trying them out. Once you are sure you set them correctly, add them to the run script “dc.tcl” so you can work in batch mode:

Constraint	Value
Clock period	10 ns
Clock setup uncertainty	10%
Clock transition	10%
Clock source latency	5%
Clock network latency	3%
Input delay *	40%
Output delay	50%
Output load	0.04 pF
Input min transition *	1%
Input max transition *	10%

* **DO NOT** add input delay nor transition to clock ports
(use the command `remove_from_collection`)

Tips:

- ✓ Create a **variable for the clock period** (something like `set clk_val 30`).
- ✓ Express the other constraint values as functions of that clock period variable using the TCL command `expr` (something like `[expr $clk_val*0.1]`).
- ✓ Check that the output for each constraint command gives no errors (should be ‘1’).

4. Pre-compile Reports

- Generate some pre-compile reports, which will report on the **constrained GTECH (generic technology) netlist**. Save them using the “>” character (e.g. `report_clock > reports/report_clock.rpt`).
- Use the following commands to report and check the constraints you just placed.
`report_clock` (period and clock source)
`report_clock -skew` (uncertainty, transition, latency)
`report_port -verbose` (input/output delays, output load, input transitions)
- Use `check_timing` to check for possible timing problems, `check_design` to check for issues in the design.

Part C: Synthetizing, reporting and saving the design

5. Compile/Synthesis

- Synthesize your design. Disable the auto-ungrouping DC option to preserve hierarchies:
`compile_ultra -no_autoungroup`
- Review the compile log displayed on screen to understand the optimization steps and cost minimization process performed by Design Compiler during synthesis.
- Open the GUI and review the schematic of the mapped gate-level netlist. **How is it different from the pre-compile unmapped generic netlist?**

6. Post-compile Reports

- Generate the typical post-compile reports (QoR, timing, area, power, constraints, etc) for your **synthesized design (gate-level netlist, mapped to the tech lib)**. For example:
 - `report_qor`
 - `report_constraints -all_violators`
 - `report_timing`
 - `report_power`
- Analyze the post-synthesis reports, take note of the relevant numbers. Does the synthesized design **meet all design rule constraints and user-defined optimization constraints**?

7. Save Design

- Save the current compiled design using the two formats: DDC and Verilog+SDC. You can name your files “mapped” (e.g. mapped.ddc, mapped.v, etc.).
- In Linux shell, compare the unmapped generic netlist (“unmapped.v”) to the synthesized gate-level netlist (“mapped.v”).
- Finally, add `exit` at the end of “dc.tcl” to close the DC shell.

Part D: Playing with constraints

- Modify constraints to see their effect on Design Compiler’s optimizations:
 - **Change max. output load to 0.001 pF** (report timing for a specific output path before and after the change)
 - **Change max. output load to 1 pF** (report timing for that same output path)
 - **Change maximum input transition max to 30% of the clock period** (report timing for a specific input path before and after the change)
 - **Change input delay to 90%.**

After each modification, run `compile_ultra -no_autoungroup` (DC will synthesize the design from scratch). See how Design Compiler optimized the design differently, start the GUI to see the schematic of the gate-level netlist, and check the `report_timing`, `report_constraints`, and `report_qor` reports to analyze results.

- Re-run the flow with a **clock period of 3 ns** (nothing else changed).